Laguna S 2.1 Performance Research & Engineering Plan

Prepared: 11 October 2026

Hardware and goals

- GPU: NVIDIA RTX 4060 Ti, 16 GB VRAM
- CPU: AMD Threadripper 1950X, 16 cores / 32 threads, AVX2
- RAM: 128 GB DDR4-2133, four memory channels
- Measured CPU↔GPU transfer bandwidth: approximately 6.6 GB/s
- Current at 8K: 779 tok/s prefill, 16.1 tok/s generation
- Targets: 900 tok/s prefill, 30 tok/s generation, eventually near 128K context
- Model file label: `UD-IQ3_S`, but expert tensors reportedly mix IQ2_S, IQ4_XS, and Q6_K
- DFlash speculative decoding enabled

Evidence labels: Established = supported by supplied measurements or cited project/model references. Estimate = calculation from stated assumptions. Hypothesis = requires a local benchmark.

Executive summary

Prefill needs +15.5% throughput (779 to 900 tok/s). Generation needs +86.3% throughput, or a reduction from about 62.1 ms to 33.3 ms per output token. The prefill target is the more focused optimization. The decode target likely needs several improvements in expert residency, CPU expert execution, and speculative decoding.

At 128K, global-attention KV memory competes directly with GPU expert-cache capacity. Quantized KV and compact sliding-window allocation should be tested before assuming decode performance will remain stable.

Highest-value next step: collect a per-token critical-path trace with CPU gate/up and down time, expert-cache hit rate, H2D copy overlap, global/sliding attention time, and DFlash accepted tokens per verification window.

1. Architecture comparison and feasibility

Projects and model identity

- Niko1221/Strata: consumer-focused Strata inference project.
- ro99/strata: separate C++/CUDA engine that documents Laguna support.
- Poolside/Laguna-S-2.1: Laguna model reference.
- Qwen/Qwen3.8-Flash-Next: Qwen model card matching the informal “Qwen3.8 Flash Next” label. Confirm the local GGUF conversion, tokenizer, and metadata before assuming exact equivalence.

Treat the two Strata repositories as separate implementations. The Qwen engine is a reference for transferable techniques; the ro99 engine is a separate reference for Laguna-specific support.

Mechanisms worth inspecting in Strata

1. **Expert residency and cache placement.** Keep a reusable expert working set in VRAM; serve misses from host RAM or a supported file tier. Profile-guided placement may improve hit rate.
2. **Asynchronous staging.** Queue future expert transfers while useful GPU work or CPU expert calculations proceed.

3 **Quantized matrix multiplication.** Prefer kernels that operate on the actual packed quantization instead of expanding full expert matrices to FP16 before multiplication.

4 **Routing-aware scheduling.** Use expert-frequency and route data to stage likely experts. Approximate skipping of uncached experts changes the computation and must not be copied as if it were exact.

5 **Speculative decoding.** Batch target verification of draft tokens. Strata's Qwen MTP path is model-specific; Laguna DFlash has different draft/feature plumbing.

Architecture comparison

Property	Qwen3.8-Flash-Next	Laguna S 2.1
Total / active parameters	125B / 6B, per model card	118B / 8.1B, per reported config
Routed experts	512	256
Experts selected per token	10	10
Expert intermediate width	640	1,024
Attention	36 Gated DeltaNet + 12 Qwen Sparse Attention layers	36 sliding-window + 12 global-attention layers
Speculation	One-layer MTP	DFlash block drafter

Model configuration and conversion differences should be checked against the actual files before treating this table as exact for a local build.

Dimension-derived expert work estimate

For a three-projection expert (gate, up, down), Laguna dimensions imply:

```
3 × 3072 × 1024 = 9.44 million projection weights per expert
```

Qwen dimensions imply:

```
3 × 2560 × 640 = 4.92 million projection weights per expert
```

That is approximately **1.92× as many projection weights per selected Laguna expert**. This is a shape-derived estimate, not a prediction of the full latency ratio. Attention architecture also matters: Qwen's recurrent/sparse attention design differs from Laguna's global plus sliding-window attention.

Is 30 tok/s physically plausible?

At 16.1 tok/s, output-token time is about 62.1 ms. At 30 tok/s it must be 33.3 ms, removing about 28.8 ms per token.

Using approximately 47 routed layers, top-10 routing, and Laguna dimensions:

```
47 × 10 × 3 × 3072 × 1024 ≈ 4.44 billion routed projection weights/token
```

Assuming 3–4.2 effective bits per routed weight gives roughly 1.66–2.33 GB of selected routed data per token before detailed packing and metadata overhead. At 30 tok/s that is 50–70 GB/s if all selected weights were read from host RAM every token. This is a sensitivity estimate, not observed traffic. The measured 6.6 GB/s CPU↔GPU transfer rate cannot support uploading all selected weights on demand at this rate.

Verdict: 30 tok/s is not ruled out by this calculation, but is a stretch target. It likely depends on high expert-cache reuse, efficient CPU execution for misses, useful overlap, and strong DFlash acceptance.

2. Ranked engineering plan

Rank	Change	Value / risk	Evidence needed
1	Expert residency and route-aware cache placement	Potentially high decode value; low/medium risk	Per-layer route IDs, hit rate, H2D bytes, cache stalls

Rank	Change	Value / risk	Evidence needed
2	Batch CPU expert work across verification rows	Potentially high if experts repeat; higher effort	Actual repeated expert IDs and weight traffic avoided
3	Tune DFlash	Potentially high, workload-dependent	Draft time, verification time, accepted tokens/window, end-to-end tok/s
4	Overlap cache uploads with CPU work	Contained experiment; gain unknown	GPU timeline proving copies overlap useful CPU work
5	Prefill chunk size and quantized GPU kernels	Best first lever for 900 tok/s	2K/4K/8K sweep, quant dispatch, peak VRAM
6	Zen 1-specific CPU kernels	Unknown until isolated	Per-kernel benchmarks; AVX2/FMA3 constraints
7	Long-context KV and attention	Required for 128K	KV allocations, attention timing, cache capacity

Prefill benchmark

Use identical model, prompt tokens, cache warmup, KV format, output settings, and build. Compare current chunk size, 2K, 4K, and 8K if workspace permits. Record throughput, peak VRAM, and actual kernel dispatch for IQ2_S, IQ4_XS, and Q6_K tensors.

The target improvement is 15.5%. Per-token prefill time needs to fall from approximately 1.284 ms to 1.111 ms.

CPU expert batching

The most interesting alternative to repeated gate/up fusion is reusing expert weights across token rows in a speculative verification batch. If several rows select the same expert, group them into a batched operation so one packed weight set serves multiple rows.

Under a simplified uniform independent-routing assumption with 256 experts and top-10 routing, expected distinct experts across `m` rows are:

$$E[U] = 256 \times (1 - (246/256)^m)$$

Verification rows	Selections	Expected distinct experts	Approx. selection reuse
8	80	70	13%
16	160	121	25%

This is a mathematical illustration only. Actual routing may be much more or less concentrated. Capture real route IDs before implementing batching.

Upload-overlap experiment

- **A:** current uploads queued after CPU expert computation.
- **B:** enqueue refresh uploads after cached GPU weight reads are submitted and before CPU completion polling, with explicit event dependencies.
- Use the existing request, fixed 8K prompt, greedy decoding, fixed output length, warmed cache, and at least 20 paired runs if practical.
- Alternate A/B runs to reduce thermal and cache-warmup bias.
- Record median tok/s, per-token latency, CPU compute/wait, H2D bytes/time, GPU idle intervals, cache hit rate, and DFlash acceptance.

CUDA ordering and lifetime rules:

- Record an event after the last GPU use of the old cache slot; make the upload stream wait before overwriting it.
- Keep pinned host source buffers allocated and unchanged until async copy completion.
- Do not publish new cache metadata until upload completion; consumers must wait for the ready event.

- Use a dedicated upload stream and asynchronous copies. Host synchronization can eliminate overlap.
- DMA and CPU kernels may contend for host-memory bandwidth; transfers may also contend for GPU memory bandwidth.

Perfect overlap would reduce combined time from `C + U` to `max(C, U)`, where `C` is CPU work and `U` is serial upload time. Real gains will be lower if the paths contend. Do not assume overlap helps unless the timeline proves it.

CPU kernel priorities

The supplied profile says CPU gate/up and down calculations dominate the CPU expert path. Worker wakeup and phase boundaries account for about 0.26% of measured phase time, and two paired IQ2 gate/up kernel candidates did not improve throughput. That makes more wakeup tuning or repeating the same fusion idea low priority. Threadripper 1950X supports AVX2/FMA3, not AVX-512. Focus on weight reuse across rows and quantify memory bandwidth before hand-tuning another kernel.

3. 128K context: memory and latency budget

KV calculation

Assumptions: 48 layers, eight KV heads, head dimension 128, 12 global-attention layers, 36 sliding-window layers, BF16 K and V, one sequence, and compact 512-token sliding-window storage.

Per layer/token:

```
2 (K and V) × 8 heads × 128 dimensions × 2 bytes = 4,096 bytes = 4 KiB
```

The 12 global layers use 48 KiB per context token. The 36 sliding-window layers use `36 × 4 KiB × 512 = 72 MiB` once full.

Context	Global KV BF16	Sliding KV BF16	Total target KV
8,192	384 MiB	72 MiB	~456 MiB
131,072	6 GiB	72 MiB	~6.07 GiB

A Q8-style cache may use roughly half BF16 storage: approximately 3.1–3.2 GiB at 128K. A Q4_0-style cache may be roughly one quarter before scales/overhead, so plan approximately 1.6–1.8 GiB and measure the actual runtime allocation and quality.

Illustrative 16 GB VRAM budget

Allocation	8K	128K
Target non-routed weights	3.5–6.5 GiB estimated	Same estimate
DFlash weights, if GPU resident	2.2 GiB	2.2 GiB
Target KV, BF16	0.45 GiB	6.07 GiB
Runtime/workspace allowance	1–2 GiB estimated	1–2 GiB estimated
Residual for expert cache with BF16 KV	~4.9–8.9 GiB	~0–3.2 GiB
Residual for expert cache with Q8-style KV	~5.1–9.1 GiB	~2.1–6.1 GiB

This is only a planning budget. It excludes possible draft KV/scratch, fragmentation, and safety margin; non-routed weight size depends on actual tensor types. The high end of the residual must not be treated as guaranteed free VRAM. Startup logs must settle the actual allocation.

Context growth

- Global-attention KV memory: linear with context; directly competes with expert cache.
- Global-attention decode reads: approximately linear per query in history length.
- Sliding-window KV: should cap at 512 tokens/layer if compact allocation works.
- Sliding-window attention work: bounded by the window per query.

- Expert MLP work per target token: roughly constant for a fixed model and route count.
- Exact full global-attention prefill work: quadratic in total prompt length; chunking reduces workspace pressure but does not remove the underlying attention work.
- DFlash weights: approximately constant; draft KV/scratch depend on implementation and block size.

A 131,072-token prompt at 900 tok/s would take about 146 seconds. Sustaining 900 tok/s over a full 128K prefill is an aggressive stretch goal. Benchmark fresh 8K, 32K, and 128K prompts separately from a short append at an already-long context.

Required runtime capabilities

- Correct FlashAttention paths for global and sliding-window layers.
- Compact sliding-window KV allocation.
- Quantized target KV support; test Q8 first, then Q4 if required and quality permits.
- Chunked prefill with controlled workspace and no unnecessary serialization of expert transfers.
- VRAM accounting for target weights, target KV, DFlash weights/state/scratch, and expert cache.
- Separate visibility into target and draft KV allocations.

4. Measurement checklist and decision gates

Measurement	Why it matters
Exact engine repo, commit, build flags	Establishes active implementation and kernel dispatch
GGUF tensor type/size inventory	Filename alone does not describe mixed expert quantization
Startup VRAM breakdown at 8K and 128K	Shows context-dependent allocation and expert-cache budget
Per-layer expert IDs and cache hit/miss counts	Distinguishes cache policy from CPU execution bottlenecks
CPU gate/up, down, attention, and wait times	Finds the actual serial critical path
H2D bytes, duration, and overlap timeline	Tests upload-overlap hypothesis
DFlash draft/verify time and accepted tokens/window	Measures speculation efficiency
Disk reads and hard faults	Verifies whether active weights remain resident in RAM

Staged plan

1. Freeze model files, engine commit, build flags, prompt tokens, cache warmup, generation length, and DFlash settings. Report median and p90.
2. Reach 900 tok/s at 8K prefill by sweeping chunk size and checking quantized kernel dispatch.
3. Trace expert routing/cache behavior; tune placement and DFlash; only then build CPU batching if route reuse justifies it.
4. Run the upload A/B with a GPU timeline and correct slot/buffer events.
5. Test 128K with Q8 KV, then Q4 if needed; verify compact sliding-window allocation and measure expert cache.
6. If CPU expert computation remains the dominant serial stage and cache hit rate is already high, stop expecting upload ordering alone to close the generation gap.

5. Primary source register

- Niko1221/Strata — Strata source, benchmark history, cache controls, staging and speculation.
- ro99/strata — separate engine and Laguna implementation reference.
- Poolside/Laguna-S-2.1 — model release and GGUF information.
- Qwen/Qwen3.8-Flash-Next — model identity and architecture.

- llama.cpp — reference quantized CPU/GPU kernels and runtime.
- AMD Threadripper 1950X — CPU capabilities.
- NVIDIA RTX 4060 Ti — GPU product reference.

Evidence classification

Supplied measurements: 779 tok/s prefill, 16.1 tok/s generation, approximately 6.6 GB/s measured CPU↔GPU transfer, CPU gate/up and down dominate CPU expert path, worker/phase overhead about 0.26%, and two paired IQ2 gate/up candidates did not improve throughput.

Source/config-dependent facts: model dimensions, layer pattern, Strata features, DFlash behavior, and CPU instruction-set capabilities. Verify against exact local files and commit.

Estimates: KV memory, routed weight traffic, expert projection sizes, throughput ratios, and residual VRAM budgets.

Hypotheses: gains from expert batching, upload overlap, route-aware cache policy, specific chunk sizes, and whether 30 tok/s or 900 tok/s remains achievable at 128K.

Final recommendation

1. **First:** reach 900 tok/s prefill at 8K through quantized GPU kernel dispatch and chunk-size experiments.
2. **Second:** pursue 30 tok/s decode through measured expert residency, DFlash acceptance tuning, and CPU batching only if route reuse is demonstrated. Run upload overlap as an isolated, low-risk A/B.
3. **Third:** establish a quantized-KV 128K configuration and measure the resulting expert-cache budget before optimizing long-context decode.

The key unresolved fact is whether the current 16.1 tok/s is limited primarily by CPU expert execution, cache misses, or net DFlash cost. A per-token critical-path trace with cache and acceptance data determines the next worthwhile change.